Algorithmique Avancée — Devoir de Programmation : Huffman Dynamique

Devoir à faire en binôme. Soutenance en séance de TD/TME 9 (semaine du 8 décembre). Rapport (au format pdf, d'une dizaine de pages), Code Source (avec vos jeux de données test), et transparents de Présentation à déposer dans une archive compressée (format zip ou tar.gz) sur `moodle` *au plus tard le 16/12 à 23h59.*

Langage de programmation libre.

Principe : Pour rappel, la méthode de Huffman compresse un texte de façon optimale en codant les caractères les plus fréquents par les codes les plus courts. L'algorithme de Huffman dynamique construit un arbre de Huffman adaptatif qui évolue au fur et à mesure de la lecture et du traitement (compression ou décompression) des symboles. L'algorithme de compression (compresseur) et l'algorithme de décompression (décompresseur) modifient l'arbre de la même façon, de telle sorte qu'à chaque instant ils utilisent les mêmes codes (mais ces codes changent au cours du temps).
Initialement le compresseur démarre avec un arbre vide : aucun symbole n'a encore reçu de code. Le premier symbole lu est transmis avec son codage initial (UTF-8). Ce symbole est alors ajouté à l'arbre et reçoit ainsi un code (de longueur variable). La prochaine fois que ce symbole sera rencontré, le compresseur transmettra son code actuel et incrémentera sa fréquence. Il se peut alors que l'arbre ne soit plus un arbre de Huffman adaptatif (AHA) et si c'est le cas, le compresseur le modifie, ce qui implique une modification des codes.
Le décompresseur travaille de la même façon : lorsqu'il lit le code initial d'un symbole UTF-8, il l'ajoute à l'arbre et lui affecte ainsi un nouveau code ; lorsqu'il lit un code de longueur variable, il utilise l'arbre courant pour déterminer le symbole correspondant, puis il modifie l'arbre de la même manière que le compresseur.
Il faut bien sûr que le décompresseur sache si le code qu'il lit est un codage initial UTF-8 ou un code de longueur variable (représentant un synmbole déjà vu) : pour cela chaque codage initial est précédé d'un code spécial d'échappement, de longueur variable. Ainsi lorsque le décompresseur lit ce code d'échappement, il sait qu'il faut lire un symbole en codé avec UTF-8 qui n'est pas encore apparu dans le texte compressé lu jusque là.
La valeur du code spécial d'échappement évolue aussi au cours du traitement : pour ce faire, on ajoute à l'arbre une feuille spéciale #, de fréquence 0, dont la position dans l'arbre varie au cours du temps.

Modification de l'arbre : On rappelle les propriétés suivantes :
Propriété 1 *Soit H un AHA avec n feuilles et $n-1$ nœuds internes. Dans la numérotation hiérarchique GDBH* GaucheDroiteBasHaut *$x_1, x_2, \ldots, x_{2n-1}$ des nœuds, on a :*

1. *$W(x_1) \leq W(x_2) \leq \ldots \leq W(x_{2n-1})$, où $W(x_i)$ est le poids du nœud x_i.*
2. *Pour $1 \leq i \leq n-1$, les nœuds x_{2i-1} et x_{2i} ont le même parent dans l'arbre.*

Propriété 2 *Etant donné un AHA et une feuille f, de numéro x_{i_0}, dont le chemin à la racine est $\Gamma_f = x_{i_0}, x_{i_1}, \ldots, x_{i_k}$ ($i_k = 2n-1$), alors l'arbre résultant de l'incrémentation de $W(f)$ sera encore un AHA ssi $W(x_{i_j}) < W(x_{i_j+1})$, pour $0 \leq j \leq k-1$. On dira dans ce cas que tous les nœuds du chemin Γ_f sont incrémentables.*

L'algorithme de modification ainsi que celui de traitement, appelé par le premier, sont présentés dans le cours à partir du transparent 47.

Le premier algorithme effectue la modification de l'AHA H après lecture du symbole s. Il produit en résultat un AHA H' dans lequel la fréquence de s a été augmentée de 1, et les autres poids ont été modifiés en conséquence.
En plus des primitives classiques sur les arbres, on devra disposer d'une fonction `finBloc(H,m)` qui, étant donné un nœud m numéroté x_m dans un AHA H, renvoie le nœud b tel que $W(x_m) = W(x_{m+1}), \ldots = W(x_b)$ et $W(x_b) < W(x_{b+1})$.

1 Introduction

Question 1 Montrer que l'algorithme de modification de l'arbre n'échange jamais un nœud avec un de ses ancêtres.

Question 2 Quel est le nombre maximal d'échanges réalisés lors d'une modification de l'arbre (en fonction de la taille de l'alphabet des symboles) ?

2 Implantations d'outils

À travers ce projet, les algorithmes de Huffman adaptatif seront implantés pour compresser des fichiers textuels encodés en UTF-8. Nous allons utiliser des fichiers binaires [1] afin de stocker le résultat des textes compressés. Concernant l'écriture et la lecture dans les fichiers binaires, nous allons utiliser les conventions suivantes : on va lire, ou écrire dans un fichier, octet par octet, donc par paquets de 8 bits. La lecture de gauche à droite se fera en lisant les bits de poids les plussforts vers les plus faibles. de l'octet. Par ailleurs, si le nombre de bits à stocker, n'est pas un multiple de 8, on complétera les bits de poids les plus faibles du dernier octet par des 0. Ainsi, si l'on souhaite écrire la suite de bits `10011011101`, le premier octet `10011011` a son bit de poids fort 1, puis son deuxième bit de poids le plus fort 1,.... Donc le premier octet encodé correspond à $10011011 =_8 155$. Le deuxième octet est complété à droite par des 0 et est donc encodé par $10100000 =_8 160$.

Question 3 Soit `fichier.bin` le nom d'un fichier binaire. Écrire une fonction `lecture` prenant en entrée `fichier.bin` et permettant la lecture du contenu du fichier et son affichage à l'écran sous forme d'une chaîne de bits.

Question 4 Soit `fichier_chaine.txt` et `fichier.bin` deux noms de fichiers.
Le fichier `fichier_chaine.txt` est textuel et contient une suite de bits (sur une seule ligne). Écrire une fonction `ecriture` prenant en entrée `fichier_chaine.txt` et `fichier.bin` et produisant en sortie un fichier binaire nommé `fichier.bin` encodant le texte `fichier_chaine.txt` sous forme de fichier binaire.
Remarque : ce fichier doit être lisible par la fonction `lecture`.

3 Compression et Décompression

L'algorithme de compression est esquissé dans le cours (transparent 49), celui de décompression sur le transparent 60.

Vous pouvez implanter les algorithmes dans le langage de votre choix et faire appel à des bibliothèques existantes pour la gestion des bits.

Question 5 Définir les structures de données nécessaires à l'implantation des algorithmes de compression et de décompression. Rappeler la complexité des opérations sur ces structures.
Remarque : pour améliorer certaines opérations, par exemple la recherche du caractère spécial dans l'arbre de Huffman, il peut être utile de maintenir des pointeurs permettant un accès direct. Le cas échéant, les indiquer en association avec la structure de données et en tenir compte dans la présentation des complexités.

Question 6 Implanter la structure arborescente permettant de manipuler l'arbre de Huffman dynamique, ainsi que la fonction `finBloc` présentée dans le cours.

Question 7 Tester les programmes de compression et décompression sur différents jeux de tests.

1. Un fichier binaire n'est pas un fichier textuel qui contient une chaîne de caractères de 0 et de 1, cf Wikipedia.

4 Étude expérimentale

Dans cette section, une analyse expérimentale est demandée. On vous propose diverses approches pour recueillir des résultats expérimentaux. Vous pouvez en proposer d'autres, toute bonne idée sera valorisée.

Question 8 En vous basant sur le projet Gutenberg [2], par exemple, ou d'autres bibliothèques de textes, recueillir des résultats expérimentaux de la compression et de la décompression par la méthode de Huffman dynamique. Indiquer des références vers le fichiers utilisés.

Question 9 Reproduire ces expérimentations en utilisant des fichiers aléatoires par exemple (des fichiers textuels où les caractères sont générés aléatoirement). Expliquer comment les fichiers ont été générés.
Remarque : dans les fichiers aléatoires, il est probablement intéressant de faire varier la distribution de probabilité des divers caractères.

Question 10 Reproduire ces expérimentations en utilisant des fichiers qui contiennent de l'information mais pas directement sous forme de texte en langue naturelle (par exemple des fichiers .json, .py, .c, ...).

Question 11 À l'aide des résultats recueillis lors des dernières questions, proposer une analyse expérimentale de la compression et la décompression par la méthode de Huffman dynamique.

5 Format du rendu du code

Les entrées/sorties de votre code doivent avoir un format strict pour nous permettre de lancer des tests automatiques.

- Si vous utilisez un langage compilé vous devez fournir un MakeFile qui permet de compiler votre code avec la commande `make`. Si vous utilisez un langage interprété le MakeFile n'est pas nécessaire.
- Un fichier textuel (au format UTF-8) verra son contenu compressé et le résultat sera stocké dans un fichier binaire.
- Veillez à indiquer dans le README de votre projet, le langage que vous avez utilisé, sa version et le compilateur utilisé ainsi que sa version.

Votre code doit pouvoir s'exécuter avec un script bash comme décrit ci-dessous :

```
compresser nom_fichier.txt nom_fichier_compresse.huff
```

La commande commence par compiler votre code (si nécessaire), puis crée la compression du texte fourni en argument (`fichier.txt` représente le nom d'un fichier textuel qui contient un texte encodé au format UTF-8) et le stocke dans le fichier binaire `fichier_compresse.huff`. Par ailleurs, à chaque compression, le fichier `compression.txt` est mis-à-jour en ajoutant une ligne contenant

- le nom du fichier d'entrée (texte original) ;
- le nom du fichier de sortie (fichier compressé) ;
- le nombre d'octets du fichier d'entrée ;
- le nombre d'octets du fichier de sortie ;
- le taux de compression (nombre d'octets du fichier compressé divisé par le nombre d'octets du fichier original), en ne conservant que 5 décimales ;
- le temps de compression en millisecondes.

Chaque information est séparée de la suivante par un point virgule.

```
decompresser nom_fichier_compresse.huff nom_fichier.txt
```

2. Le projet Gutenberg propose de nombreux fichiers de texte en langage naturel au format UTF-8.

La commande crée la décompression du fichier binaire `nom_fichier_compresse.huff` fourni en argument et stocke le résultat dans un fichier textuel au format UTF-8.
Un fichier `decompression.txt` (analogue à `compression.txt`) est mis-à-jour à chaque décompression.

Exemple, si votre code est en python votre fichier bash pour la commande `compresser` pourrait ressembler à :

```
#!/bin/bash

# Check if exactly two arguments are provided
if [ "$#" -ne 2 ]; then
        echo "Usage: $0 <x> <y>"
        exit 1
fi

python3 ./compresser.py "$1" "$2"
```

6 Cahier des charges

Le **rapport** doit contenir les réponses aux questions posées dans l'énoncé, une justification des choix technologiques (langage, structures de données), la description de l'architecture du logiciel, un tableau de tests avec lien avec la complexité théorique des courbes et analyse des résultats.

Il faut aussi intégrer un paragraphe expliquant comment vous vous êtes aidé d'une IA générative que ce soit pour générer du code ou des parties de rapport. Faites une analyse critique de votre usage de ces outils. Et donnez une estimation quantitative de la proportion de code généré automatiquement.

Lors de la **soutenance**, chaque binôme devra expliquer son travail en 10 minutes : présentation sur 2 ou 3 transparents suivie d'une démonstration.